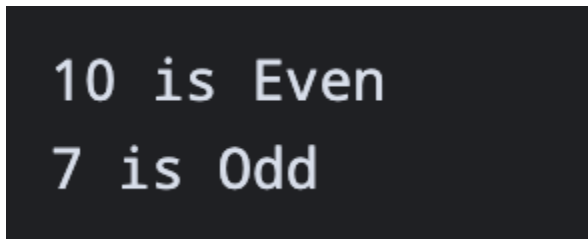


Q1.

Problem Statement: Write a JavaScript program that takes a number and checks if it is an even number or an odd number.

Source Code:

```
function checkEvenOdd(number) {  
    if (number % 2 === 0) {  
        console.log(number + " is Even");  
    } else {  
        console.log(number + " is Odd");  
    }  
}  
  
checkEvenOdd(10);  
  
checkEvenOdd(7);
```



```
10 is Even  
7 is Odd
```

Explanation

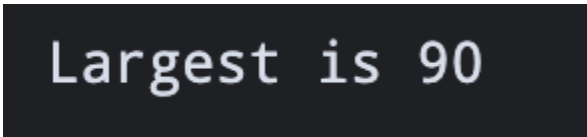
the modulo operator %, which gives us the remainder of a division. If you divide a number by 2 and the remainder is 0 the number is even Otherwise it is odd.

Problem Statement : Find the biggest number among three given numbers.

Source Code:

```
let a = 33, b = 55, c = 90;
```

```
if (a >= b && a >= c) {  
    console.log("Largest is " + a);  
} else if (b >= a && b >= c) {  
    console.log("Largest is " + b);  
} else {  
    console.log("Largest is " + c);  
}
```



Largest is 90

Explanation:

compare each number with the other two and get the number which greater than or equal to both others is the largest.

Problem Statement: Write a JavaScript program that takes a string of text and reverses the order of it

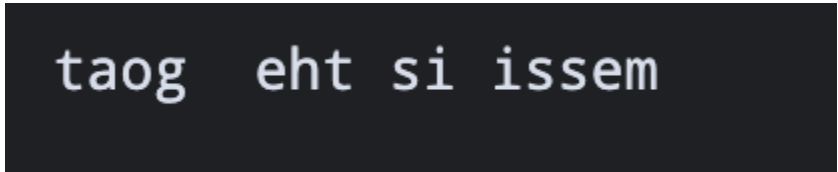
Source Code:

```
let str = " messi is the  goat";
```

```
let reversed = "";
```

```
for (let i = str.length - 1; i >= 0; i--) {  
    reversed = reversed + str[i];  
}
```

```
}  
  
console.log(reversed);
```



```
taog eht si issem
```

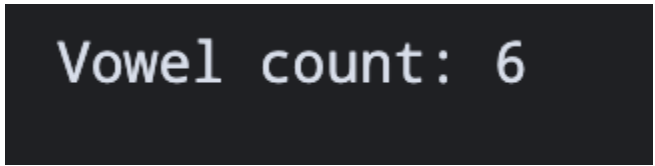
Explanation:

start a loop from the last character and go doing backtracking adding each character to a new string one at a time one by one.

Problem Statement: Write a JavaScript program that counts how many vowels exist

Source Code:

```
let str = "messi is the goat";  
  
let vowels = "aeiou";  
  
let count = 0;  
  
for (let i = 0; i < str.length; i++) {  
    if (vowels.includes(str[i])) {  
        count++;  
    }  
}  
  
console.log("Vowel count: " + count);
```



```
Vowel count: 6
```

Explanation:

check each character of the string. If it exists inside the vowels string, and increase the counter.

Problem Statement: Write a JavaScript program that takes an array with repeating values and remove the duplicates

Source Code:

```
const numbers = [544,510,213,510, 510, 344,333, 544, 333, 1];
```

```
const uniqueNumbers = numbers.filter((value, index, array) => {  
    return array.indexOf(value) === index;  
});
```

```
console.log(uniqueNumbers);
```

```
[ 544, 510, 213, 344, 333, 1 ]
```

Explanation

create a new empty array for every value in the original array only add it to the new array if its not there

Q2.

What is Node.js?

Node.js is an open-source, cross-platform javascript runtime environment built on Google Chrome's highly optimized V8 JavaScript engine.

Traditionally, JavaScript could only run inside a web browser to make websites interactive. Node.js changed this by providing an environment where JavaScript can run directly on a computer's operating system giving it access to file systems, databases, and network requests.

How Does Node.js Work?

Node.js works using a few key ideas:

1. V8 Engine – Converts your JavaScript code into fast machine code that the computer understands.
2. Event Loop – This is the heart of Node.js. Instead of waiting for slow tasks (like reading a file or calling an API) to finish, Node.js sends that task aside, keeps running other code, and comes back to it once it's done.
3. Non-blocking – Because of the event loop, Node.js doesn't "freeze" while waiting for slow operations. This makes it fast and efficient.

Advantages of Node.js for QA Automation

Node.js comes with several benefits for QA automation, including the ability to provide a uniform tech stack. When developers work on web applications in JavaScript or TypeScript, QA specialists have an opportunity to automate their tests with the very same programming language, which facilitates the interaction between both sides. Also, Node.js allows you to leverage the benefits of NPM, the biggest software registry in the world, and use the best-in-class automation tools such as Playwright, Cypress, and WebdriverIO. Since UI testing is performed in an asynchronous manner, because it involves continuous waiting for page elements to load and for API responses to be received, the Node.js event-driven architecture and async/await syntax are designed to manage these delays perfectly.

Difference Between Browser JavaScript and Node.js

Browser JavaScript and Node.js are both based on the same JavaScript language, but they are used in very different environments and for different purposes. Browser JavaScript runs inside a web browser such as Chrome or Firefox, and its main job is to make web pages interactive — for example, validating forms, showing animations, or responding to button clicks. It has access to the DOM(Document Object Model), meaning it can directly interact with documents window,

and other HTML elements on a webpage. However, for security reasons, it cannot read or write files on the user's computer. Node.js, on the other hand, runs outside the browser, directly on a computer or server, and is mainly used to build backend applications, servers, and automation scripts. Unlike browser JavaScript, Node.js has no DOM access and cannot interact with HTML elements, but it can read and write files using its built-in modules, which browser JavaScript cannot do. Node.js also uses `require()` or `import` statements to bring in external packages from npm, whereas browser JavaScript typically relies on `<script>` tags or ES modules. In simple terms, browser JavaScript is used to make webpages interactive for users, while Node.js is used to build the tools, servers, and automated processes that work behind the scenes.